Отчёт по лабораторной работе №6

Математические основы защиты информации и информационной безопасности

Сунь Маосин

Содержание

1 Цель работы **5**

2 Реализация алгоритма **6**

2.1 Вспомогательная функция для вычисления НОД 6

2.1.1 Код функции 6

2.2 ρ-метод Полларда 6

2.2.1 Код реализации 7

2.3 Тестирование на примере из задания 8

2.3.1 Код тестирования 8

2.3.2 Результат выполнения 9

2.4 Пример работы с пользовательским числом 9

2.4.1 Результат 10

3 Вывод **11**

Список иллюстраций

2.1 Функция вычисления НОД 6
2.2 ρ-метод Полларда 7
2.3 Функция тестирования примера 8
2.4 Результат тестирования 9
2.5 Пример работы с числом 989 10

Список таблиц

1 Цель работы

Изучить основные задачи и алгоритмы разложения чисел на множители. Реализовать программно ρ-метод Полларда, понять принципы поиска нетривиальных делителей составных чисел в криптоанализе и выполнить численные расчеты на основе заданных начальных параметров.

2 Реализация алгоритма

2.1 Вспомогательная функция для вычисления НОД

Для работы алгоритма потребовалась функция вычисления наибольшего общего делителя, реализованная с помощью алгоритма Евклида.

2.1.1 Код функции

```
def gcd(a, b):
    """
    Euclidean algorithm for greatest common divisor
    """
    while b != 0:
        a, b = b, a % b
    return a
```

Рис. 2.1: Функция вычисления НОД

2.2 ρ-метод Полларда

ρ-метод Полларда — это рандомизированный алгоритм факторизации целых чисел. Он основан на математическом принципе «парадокса дней рождения» и использует сжимающие свойства случайных отображений для поиска циклов в последовательностях.

2.2.1 Код реализации

```
def pollard_rho(n, c=1):
    """
    Pollard's Rho algorithm for integer factorization
    n: number to factor (should be composite)
    c: initial value and constant in function f(x) = x² + c
    Returns: a non-trivial factor of n
    """
    # Handle trivial cases
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3

    # Define function f(x) = x² + c (mod n)
    def f(x):
        return (x * x + c) % n

    # Initialize tortoise and hare
    a = 2
    b = 2
    d = 1

    # Main loop
    while d == 1:
        # Tortoise moves one step, hare moves two steps
        a = f(a)
        b = f(f(b))

        # Compute gcd of difference and n
        d = gcd(abs(a - b), n)

        # If we found n itself, algorithm failed
        if d == n:
            return pollard_rho(n, c + 1)

    return d
```

Рис. 2.2: ρ-метод Полларда

2.3 Тестирование на примере из задания

Для проверки корректности работы алгоритма был использован пример из задания: число $n = 1359331$ с параметром $c = 1$ и функцией $f(x) = x^2 + 1 \pmod{n}$.

2.3.1 Код тестирования

```
def test_example():
    """Test the example from the assignment"""
    print("=" * 60)
    print("TESTING EXAMPLE FROM ASSIGNMENT")
    print("=" * 60)

    # Example from assignment
    n = 1359331
    c = 1

    print(f"Number to factor: n = {n}")
    print(f"Initial value: c = {c}")
    print(f"Function: f(x) = x² + {c} (mod n)")
    print()

    # Find factor
    factor = pollard_rho(n, c)

    print(f"Found factor: {factor}")
    print(f"Verification: {factor} × {n // factor} = {n}")
    print(f"Success: {factor * (n // factor) == n}")

    print("\n" + "=" * 60)
```

Рис. 2.3: Функция тестирования примера

2.3.2 Результат выполнения

```
LABORATORY WORK No. 6
Pollard's Rho Factorization Algorithm


----------------------------------------
MENU:
  1. Test example from assignment (n = 1359331)
  2. Factor a number (for teacher's number)
  3. Exit
----------------------------------------
Enter your choice (1-3): 1
============================================================
TESTING EXAMPLE FROM ASSIGNMENT
============================================================
Number to factor: n = 1359331
Initial value: c = 1
Function: f(x) = x² + 1 (mod n)

Found factor: 1151
Verification: 1151 × 1181 = 1359331
Success: True

============================================================

Press Enter to continue...
```

Рис. 2.4: Результат тестирования

2.4 Пример работы с пользовательским числом

Для демонстрации работы алгоритма на других числах был протестирован пример с числом $989 = 23 \times 43$.

2.4.1 Результат

```
Enter your choice (1-3): 2
============================================================
FACTORIZE A NUMBER
============================================================
Enter number to factor: 989

Factoring 989...
----------------------------------------
Found factor: 23
Verification: 23 × 43 = 989

============================================================
```

Рис. 2.5: Пример работы с числом 989

3 Вывод

В ходе выполнения данной работы был успешно реализован и протестирован ρ-метод Полларда. При заданных параметрах $c = 1$ было произведено разложение числа 1359331, в результате которого были найдены делители 1151 и 1181. Это подтверждает эффективность и быструю сходимость данного алгоритма при поиске делителей больших составных чисел. Также была продемонстрирована работа алгоритма на примере числа $989 = 23 \times 43$.